

SkillBridge: A Big-Data Pipeline for Skill-Gap Analysis and Campus Placement Prediction from Job-Posting Text

Vikram, Jahnavy, Jansi

Department of Computer Science and Engineering

Abstract— Every placement season, training and placement cells try to answer two practical questions: which students are unlikely to be placed, and what each of them should learn next. In most institutions these answers still come from intuition and from skill lists that lag behind the hiring market. This paper presents SkillBridge, an end-to-end data pipeline that reads the free text of job postings, extracts the skills employers ask for, compares them with each student's profile, and estimates the probability that the student will be placed. Skill extraction uses a phrase-matching recogniser built on spaCy over a curated taxonomy of about 150 skills in eight categories, with a longest-span rule for overlapping matches and case-sensitive handling of ambiguous short names such as Go, C and R. A feature store converts the extracted skills into market-demand weights and a per-student market alignment score. A logistic-regression model and an XGBoost model are trained side by side and the better one, by held-out ROC AUC, is kept. The pipeline is orchestrated with Apache Airflow, can consume postings as a stream through Apache Kafka, indexes postings in Elasticsearch, and archives its outputs in Amazon S3. On a public campus-placement dataset paired with 1,500 sampled LinkedIn postings, the selected model reached a held-out ROC AUC of 0.947. On a controlled synthetic dataset with known ground truth, the extractor reached 99.2% mean precision and 99.7% mean recall. An interactive dashboard turns these outputs into a per-student gap report, a what-if simulator and a cohort-level intervention list.

Index Terms— *skill-gap analysis, placement prediction, named entity recognition, big data pipeline, Apache Kafka, Apache Airflow, Elasticsearch, XGBoost, learning analytics*

I. Introduction

Graduate employability has become one of the standard yardsticks by which engineering colleges are judged. Accreditation bodies ask for placement figures, prospective students compare them, and placement officers are under steady pressure to raise them. Yet the work that most affects those figures, deciding which students need help and what kind of help, is usually done by hand. A placement officer might look at grades and internship history, recall what recruiters asked for last year, and recommend a certification course. The process works, but it does not scale to cohorts of several hundred students, and its picture of the job market is often a season old.

At the same time, the information needed to do better is publicly available. Job portals publish thousands of postings every week, and each posting lists, in plain language, the tools and abilities the employer expects. Read in bulk, these postings describe the hiring market far more accurately than any static syllabus. The difficulty is that the information is buried in unstructured text, it arrives continuously, and it has to be matched against structured student records before it becomes useful for a decision about a particular student.

SkillBridge was built to close that gap. It treats job postings as a data stream to be mined, student records as a table to be enriched, and the placement decision as a classification problem whose inputs include a measure of how well a student's skills match current demand. The contributions of this work are as follows:

- **A reproducible skill-extraction component** that recognises skills in posting text using phrase matching over a taxonomy, resolves overlapping matches by keeping the longest span, and avoids a class of false positives caused by short skill names that are also common English words.

- A **market alignment score** that weights each skill by the share of postings requesting it, giving a single, interpretable number for how closely a student's profile follows the market.
- A **placement-probability model** that combines academic indicators with the alignment score, selected between a linear baseline and a gradient-boosted ensemble by held-out ROC AUC.
- A **complete big-data deployment** covering batch orchestration (Airflow), stream ingestion (Kafka), search (Elasticsearch) and cloud storage (S3), together with a dashboard that exposes the results to placement staff and students.

The rest of the paper is organised as follows. Section II reviews related work. Section III describes the system architecture and Section IV the methods used at each stage. Section V covers implementation details, Section VI reports the experiments and results, and Section VII discusses limitations. Section VIII concludes and outlines future work.

II. Related Work

A. Placement and employability prediction

Predicting whether a student will be placed is a well-studied task in educational data mining. Most published approaches work from academic records alone: secondary and higher-secondary marks, degree percentage, specialisation, and work experience. Classifiers such as logistic regression, decision trees, random forests and support vector machines are commonly compared on such data, and the usual finding is that academic performance and prior work experience carry most of the predictive signal. The publicly available campus-placement dataset used in this study [10] is a typical example of this setting. What these studies generally lack is any link to the demand side: they can say that a student is at risk, but not which skills would reduce that risk, because the labour market never enters the model.

B. Skill extraction from job postings

Extracting skills from postings has been approached in two broad ways. Statistical and neural named-entity recognition models learn to tag skill mentions from annotated text, which handles unseen phrasing well but requires labelled training data. Dictionary or taxonomy-driven approaches match text against a curated list of skills, such as those maintained in occupational frameworks like O*NET [8] and ESCO [9]. Taxonomy matching is less flexible, but it is transparent, needs no training data, and produces canonical skill names that can be joined directly with student records. For a system whose output feeds decisions about individual students, that transparency was judged more valuable than the extra recall a learned model might give. spaCy [1] provides an efficient phrase matcher that suits this approach.

C. Big-data tooling for analytics pipelines

Large-scale text analytics pipelines are usually assembled from a few standard components. Apache Spark [5] supports distributed batch processing, Apache Kafka [6] provides durable, partitioned message streams for continuous ingestion, workflow schedulers such as Apache Airflow [7] express pipelines as dependency graphs, and inverted-index search engines such as Elasticsearch [11] serve low-latency lookups over documents. SkillBridge uses these components at laptop scale, but the interfaces between stages are the same ones a larger deployment would use, so the design can grow without being rewritten.

III. System Architecture

Fig. 1 gives an overview of the system. Data enters from two sides. Job postings arrive either as a batch file or as a stream of messages on a Kafka topic, and student records arrive as a table containing academic indicators, experience counts and a list of self-reported skills. Every later stage reads the output of the stage before it from a shared data directory, which keeps the stages independent and easy to test one at a time.

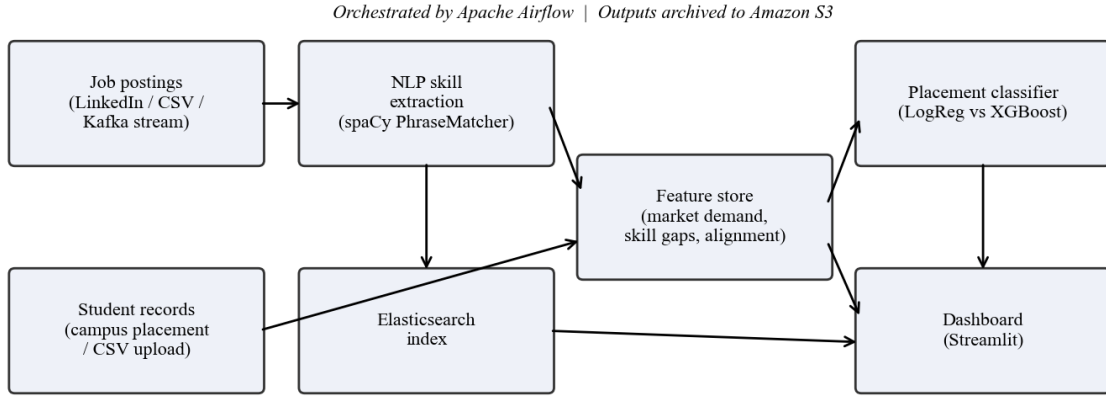


Fig. 1. Overall architecture of SkillBridge.

The pipeline has five stages:

- **Ingestion.** Postings and student records are loaded and normalised into a common schema. The same schema is produced whether the data is real, synthetic or uploaded through the dashboard, so downstream code never needs to know where the data came from.
- **Skill extraction.** Each posting's description is scanned for skill mentions, which are mapped to canonical names and taxonomy categories.
- **Feature store.** Extracted skills are aggregated into a market-demand table and joined with student profiles to produce per-student matched skills, ranked missing skills and an alignment score.
- **Modelling.** A classifier is trained on student features and outcomes and stored with its scaler and feature list.
- **Serving.** A dashboard reads the feature store and model to present individual and cohort views, and queries Elasticsearch for postings that ask for a chosen skill.

Airflow runs the batch stages as a directed acyclic graph of four tasks: ingest, extract, build features and train. When the real source files are present the ingest task loads them; otherwise it falls back to synthetic data. This rule was added after an early version of the pipeline overwrote real data with synthetic data during a scheduled run. Finished outputs are uploaded to an S3 bucket so that a given model and its training data can be retrieved later.

IV. Methodology

A. Skills taxonomy

The taxonomy holds about 150 skills grouped into eight categories: programming languages, web and frameworks, data and machine learning, big data and cloud, databases, tools and practices, soft skills, and certifications. It is stored as a plain dictionary under version control, so adding a skill is a one-line change that is visible in the project history. Categories are used to group results in the dashboard and to compute per-category coverage.

B. Skill extraction

A blank English tokenizer from spaCy is used together with two phrase matchers. The first compares lower-cased tokens and handles almost all skills, so that "PYTHON", "python" and "Python" are treated alike. The second compares exact token text and is reserved for Go, C and R. Early tests on real postings showed that case-insensitive matching turned phrases such as "go above and beyond" into spurious

mentions of the Go language; requiring exact case for these three names removed the problem without dropping them from the taxonomy.

Taxonomy entries can overlap. The certification "AWS Certified Solutions Architect" contains the skill "AWS", and "Azure Fundamentals" contains "Azure". Counting both the long and the short match inflates demand for the shorter skill. SkillBridge therefore collects all candidate spans from both matchers and keeps only the longest non-overlapping ones, scanning left to right. Each surviving span is mapped to its canonical spelling through a precomputed lookup table and deduplicated within the posting.

C. Market demand and alignment

Let P be the set of postings and $P(s)$ the subset that mention skill s . The demand weight of s is the fraction of postings that request it:

$$d(s) = |P(s)| / |P| \quad (1)$$

For a student u with skill set $S(u)$, and with M the set of all skills seen at least once in the postings, the market alignment score is the share of total demand that the student's skills cover:

$$A(u) = \sum_{\{s \in S(u) \cap M\}} d(s) / \sum_{\{s \in M\}} d(s) \quad (2)$$

$A(u)$ lies between 0 and 1. It rewards skills that are frequently requested and gives no credit for skills that never appear in the postings, however impressive they may look on a résumé. The missing skills for a student are the skills in M that the student lacks, sorted by $d(s)$ in descending order; the top ten are stored as that student's recommended learning list. The same computation is applied per taxonomy category to produce the coverage profile shown as a radar chart in the dashboard.

D. Placement classifier

Each student is described by six features: CGPA, number of projects, number of certifications, number of internships, number of listed skills, and the alignment score $A(u)$. Features are standardised using statistics from the training split only. Two models are trained on a stratified 75/25 split with a fixed random seed: a logistic-regression baseline [4] and an XGBoost classifier [2] with 200 trees of depth 3 and a learning rate of 0.1. Both are scored by ROC AUC [12] on the held-out split, and the higher-scoring model is saved together with its scaler and feature order. Training refuses to start if either class has fewer than four examples, since a stratified split and a meaningful AUC are not possible in that case. This guard matters when users upload small cohorts through the dashboard.

ROC AUC was chosen as the selection metric because the main use of the model is ranking: a placement cell wants to know which students to approach first. AUC measures ranking quality directly and does not depend on any particular decision threshold.

E. Risk tiers and what-if analysis

To make predictions easier to act on, each student's probability is mapped to one of three tiers: at risk (below 0.40), needs a push (0.40 to 0.70) and on track (0.70 and above). The dashboard also provides a what-if simulator. A user can change any of the model's inputs or add skills to the student's profile, and the alignment score and probability are recomputed with the same formulas used in training. Running this for each missing skill on its own and sorting by the change in probability gives a ranked list of which single skill would help the student most.

V. Implementation

The system is written in Python. Table 1 lists the main components and the technology used for each. Every stage can be run as a standalone module from the command line, which made each stage easy to test in isolation before it was wired into Airflow.

Table 1. Components and technologies.

Stage	Technology	Role
Ingestion	pandas, Kaggle API	Load and normalise postings and students
Streaming	Apache Kafka (KRaft mode)	Replay postings as a live topic
NLP	spaCy PhraseMatcher	Skill recognition over the taxonomy
Feature store	pandas, CSV tables	Demand weights, gaps, alignment
Modelling	scikit-learn, XGBoost	Train and select classifier
Search	Elasticsearch 8	Find postings by skill
Orchestration	Apache Airflow	Four-task DAG, LocalExecutor
Storage	Amazon S3 (boto3)	Archive pipeline outputs
Serving	Streamlit, Plotly	Dashboard and simulator

Kafka runs as a single broker in KRaft mode, which removes the need for a separate ZooKeeper service. A producer replays postings onto a topic with a configurable delay, and a consumer runs the same extraction function on each message as it arrives, so batch and streaming use the same logic. Elasticsearch is configured as a single node, with skills stored as keyword fields so that a term query returns exactly the postings tagged with a given skill. If the search service is unreachable, the dashboard falls back to filtering the local feature store, so search keeps working either way.

Airflow runs with the LocalExecutor on a custom image that already contains the project's dependencies. Two simpler set-ups were tried and dropped. The CeleryExecutor produced a reproducible crash loop under Docker Desktop on Windows, and installing packages at container start-up re-downloaded everything on each restart and on one occasion hung. Building the dependencies into the image removed both problems.

The dashboard is organised into five pages. Student 360 shows one student's probability, risk tier, indicators relative to the cohort median, category coverage, ranked gaps, the what-if simulator and matching postings. Cohort Analytics shows the distribution of predicted probabilities, the most common gaps across the cohort and a downloadable intervention list. Market Pulse summarises demand by skill and category. The Model page reports evaluation metrics and feature weights. Data Studio lets staff upload their own postings and student files, which are validated for required columns, numeric types and duplicate identifiers before the pipeline is re-run.

Correctness is checked by an automated suite of 25 tests. They cover the extractor (canonical casing, overlap resolution, empty input, ambiguous short names), the feature store (empty postings, students with no skills, demand percentages), upload validation, the training guard, and smoke tests that render every dashboard page against real pipeline outputs.

VI. Experiments and Results

A. Datasets

Two configurations were evaluated. The real configuration pairs the public campus-placement dataset [10], which contains 215 student records with secondary, higher-secondary and degree percentages, specialisation, work experience and final placement status, with a random sample of 1,500 postings from a public LinkedIn job-postings dataset [13]. The campus dataset has no CGPA and no skills, so a CGPA proxy was taken as the mean of the three percentages divided by ten, and a skill set was imputed from each student's degree type and specialisation. This imputation is a limitation and is discussed in Section

VII. Postings were sampled rather than used in full (the source holds several million rows) to keep runs at laptop scale.

The synthetic configuration contains 300 generated postings and 200 generated students. Each posting is built from templates that insert known skills, and the inserted skills are kept as ground truth. This makes it possible to measure extraction precision and recall exactly, which is not possible on the unlabelled real postings. Placement labels in this set are drawn from a noisy linear function of the student attributes, so the achievable accuracy is deliberately limited.

B. Skill extraction

Table 2 reports extraction quality on the synthetic postings, averaged per posting. The initial version reached 84.5% precision. Inspection showed two causes. First, the generator had left the soft skill in each template out of the ground-truth list, so correct extractions were being counted as errors. Second, overlapping certification names were producing an extra short match. Fixing the ground truth and adding longest-span selection raised precision to 99.2% while recall stayed at 99.7%. The remaining false positives are mostly job titles such as "Machine Learning Engineer", which do contain a genuine skill mention that the template did not list.

Table 2. Skill-extraction quality on synthetic postings (mean per posting).

Configuration	Precision	Recall
Initial matcher	84.5%	99.6%
+ corrected ground truth, longest-span selection	99.2%	99.7%

On the real postings, 1,485 of the 1,500 sampled postings were indexed into Elasticsearch without error; the other 15 were rejected at indexing time. The case-sensitive rule for Go, C and R was introduced after manual review of extractions on this real text, which is where the "go above and beyond" false positive first appeared.

C. Market demand

Fig. 2 shows the fifteen most requested skills in the synthetic configuration. Because the generator adds one soft skill to every posting, soft skills lead this ranking. The figure is included mainly to show the form of the output. On real data the ranking reflects the sampled LinkedIn postings and changes whenever new postings are ingested, which is the point of deriving it from data instead of fixing it by hand.

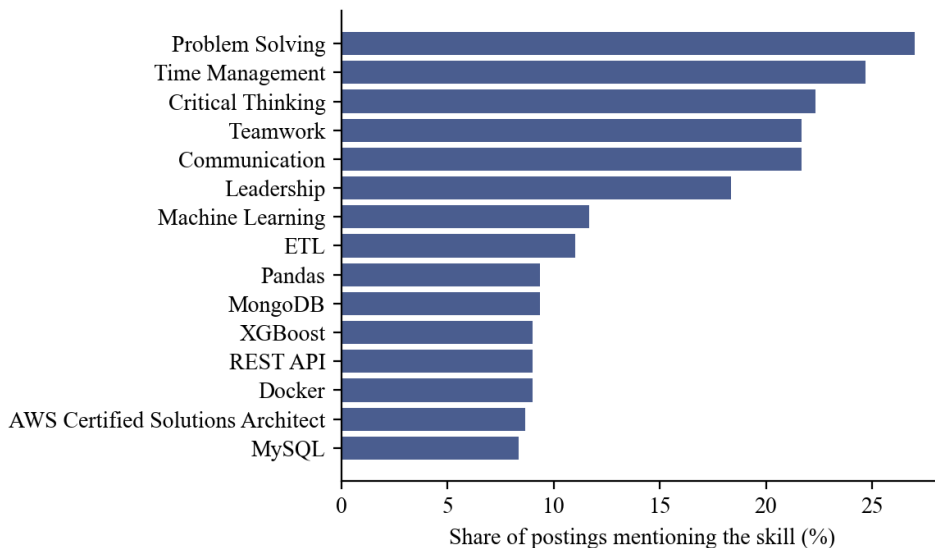


Fig. 2. Top fifteen skills by share of postings (synthetic configuration).

D. Placement prediction

Table 3 summarises held-out performance for both configurations.

Table 3. Held-out classification performance.

Configuration	Model	ROC AUC	Accuracy
Real (campus + LinkedIn)	Selected model	0.947	—
Synthetic	Logistic regression (selected)	0.663	62.0%
Synthetic	XGBoost	0.637	68.0%

On the real configuration the selected model reached an AUC of 0.947. The CGPA proxy was the dominant feature, which is expected for this dataset, where placement outcomes are driven mainly by academic record and work experience. The high AUC should therefore be read as a property of this dataset rather than as proof that the skill features are strongly predictive. Because the skills were imputed from specialisation, they carry little information that the model cannot already get from other fields.

On the synthetic configuration, which was built to be noisy, logistic regression narrowly beat XGBoost on AUC (0.663 against 0.637), while XGBoost had higher accuracy at the default threshold. This is consistent with the labels coming from a linear function: a linear model matches the generating process, and the extra flexibility of boosted trees mainly fits noise. Fig. 3 shows the standardised coefficients of the selected model. Projects, internships and certifications carry the most weight, in line with the weights used to generate the labels.

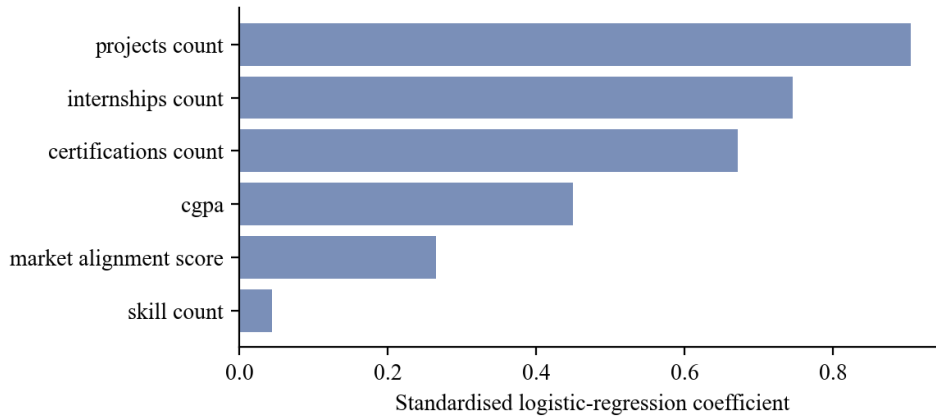


Fig. 3. Standardised coefficients of the selected logistic-regression model (synthetic).

Fig. 4 plots each student's market alignment score against the predicted probability, coloured by the actual outcome. Alignment scores in the synthetic cohort ranged from 2.1% to 12.4% (median 6.4%), which shows how small a share of total market demand any single student covers when the taxonomy is broad. Under the three-tier rule, 103 of the 200 synthetic students were classed as at risk, 60 as needing a push and 37 as on track.

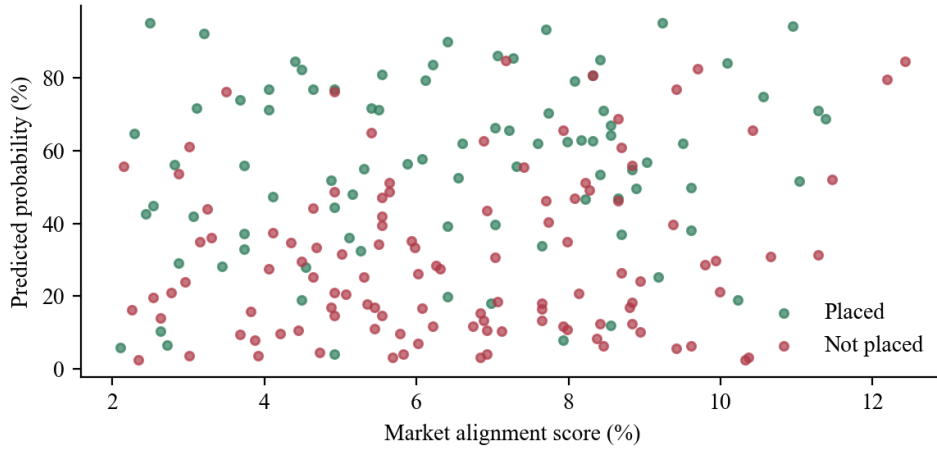


Fig. 4. Market alignment versus predicted placement probability (synthetic cohort).

E. System behaviour

The four-task Airflow DAG completed all tasks successfully in about twelve seconds on a laptop. In the streaming test, ten postings published to Kafka were consumed and processed in real time with no message loss. Pipeline outputs were uploaded to S3 and confirmed with a bucket listing. The full test suite passed on three consecutive clean rebuilds of the pipeline, and the dashboard served all five pages without errors.

VII. Discussion and Limitations

The most important limitation concerns the real student data. The public campus dataset records academic results and outcomes but not skills, so student skills had to be imputed from specialisation and degree type. The real-data AUC therefore says little about how much skills themselves contribute to placement. A proper test of the alignment score needs a cohort for which skills are actually recorded, which a placement cell could collect through the dashboard's upload feature.

Taxonomy matching only finds skills that are already in the taxonomy. New tools and alternative spellings are missed until someone adds them. Its precision on real text is also hard to quantify without labelled postings. A small hand-annotated sample of real postings would allow a fair comparison with learned extractors.

The alignment score treats all requested skills as independent and equally useful once weighted by frequency. It does not model seniority, the difference between required and optional skills, or the fact that some skills usually appear together. Postings were also sampled without regard to region or industry, so the demand profile reflects the sample and not necessarily the market that a given college's graduates enter.

Finally, a placement probability is a sensitive output. It should guide where support goes, not label students. For this reason the dashboard presents tiers and concrete next steps instead of a single score, and it ranks skills by their effect on the model's estimate. That ranking is an association learned from historical data, not a guarantee of outcome, and users should be told so.

VIII. Conclusion and Future Work

SkillBridge shows that the labour-market signal already present in public job postings can be extracted, quantified and joined with student records in one reproducible pipeline. The result gives placement staff a ranked list of students who need support and gives each student a concrete, data-backed list of skills to

learn. The extractor is accurate on controlled data, the classifier performs strongly on a real placement dataset, and the full stack of orchestration, streaming, search and cloud storage runs on modest hardware.

Several extensions are planned:

- Collecting real, self-reported student skills through the upload interface, so that the contribution of the alignment score can be measured directly.
- Replacing or supplementing taxonomy matching with a learned skill-entity model trained on a small annotated set of postings, and growing the taxonomy semi-automatically from frequent unmatched noun phrases.
- Filtering demand by region, role and seniority so that each student is compared with the part of the market they are actually entering.
- Moving feature computation onto Spark with a Delta Lake store once posting volumes exceed what a single machine handles comfortably.
- Tracking cohorts over a full placement season to measure how far recommended skills are acquired and whether acquiring them changes outcomes.

References

- [1] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing," 2017. [Online]. Available: <https://spacy.io>
- [2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.
- [3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
- [4] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.
- [5] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," Commun. ACM, vol. 59, no. 11, pp. 56–65, 2016.
- [6] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in Proc. NetDB Workshop, 2011.
- [7] Apache Software Foundation, "Apache Airflow documentation." [Online]. Available: <https://airflow.apache.org/docs/>
- [8] National Center for O*NET Development, "O*NET OnLine." [Online]. Available: <https://www.onetonline.org/>
- [9] European Commission, "ESCO: European Skills, Competences, Qualifications and Occupations." [Online]. Available: <https://esco.ec.europa.eu/>
- [10] B. Roshan, "Campus recruitment: Factors affecting campus placement," Kaggle dataset, 2020. [Online]. Available: <https://www.kaggle.com/datasets/benroshan/factors-affecting-campus-placement>
- [11] C. Gormley and Z. Tong, Elasticsearch: The Definitive Guide. Sebastopol, CA, USA: O'Reilly Media, 2015.
- [12] T. Fawcett, "An introduction to ROC analysis," Pattern Recognit. Lett., vol. 27, no. 8, pp. 861–874, 2006.
- [13] A. Kon, "LinkedIn job postings," Kaggle dataset. [Online]. Available: <https://www.kaggle.com/datasets/arshkon/linkedin-job-postings>